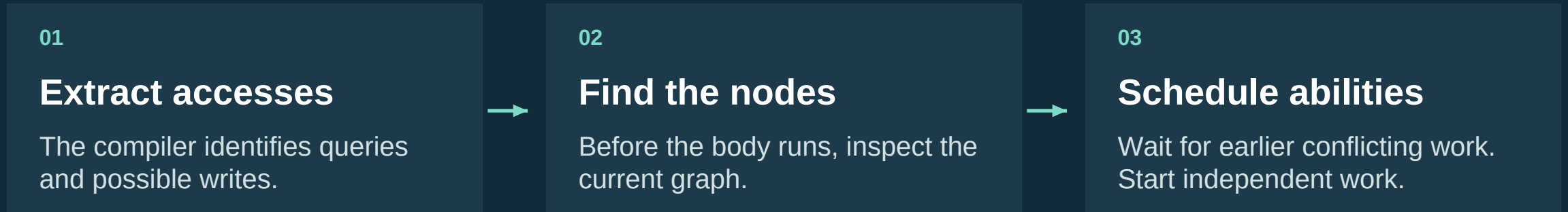


Resolve Dependencies Before Abilities Run

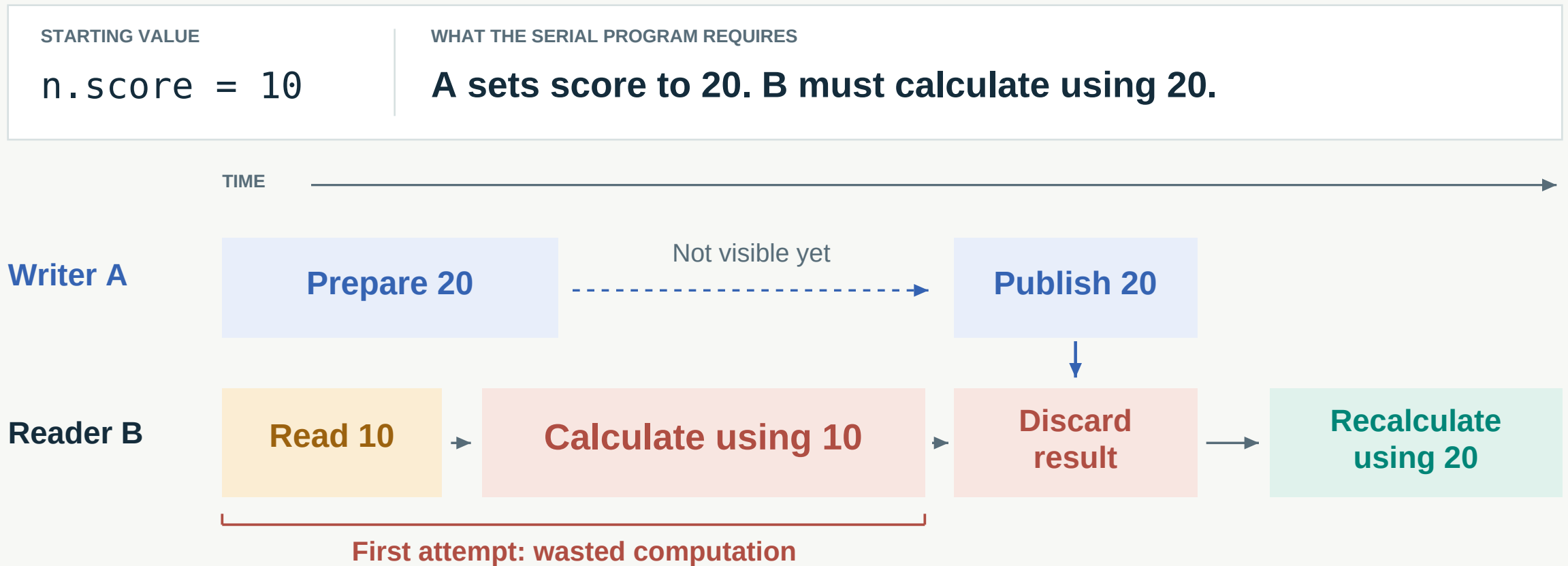
Use Jac graph queries to decide which abilities can start.



Can a small amount of inspection prevent a large amount of wasted computation?

B can finish a calculation using the wrong input.

Two abilities share a node n . The original program runs A before B.



The input was 10. The required input was 20. B has to calculate again.

Run a small inspection before the ability body.

For queries whose starting node and parameters are already known.

IN THE COMPILER

Recognize B's graph query:

`[ -> : Edge : -> ]`

Generate a separate function that finds B's possible read/write targets.

The ability body stays intact.



IN THE SCHEDULER — BEFORE B STARTS

B's current node is known.

Inspect its outgoing Edge links:

The query finds n1 and n2.

B may read these nodes.

B has not started its computation.

Jac's built-in query tells the compiler how to find the relevant nodes.

Match reads with writes before choosing what starts.

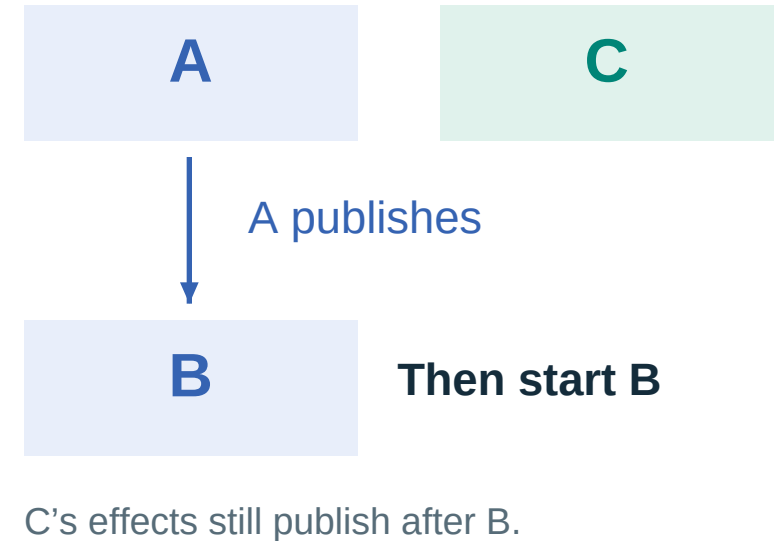
Three queued ability calls. Their original serial order is A, then B, then C.

INSPECTED NODE ACCESSES

Call	May read	May write
A	—	n2
B	n1, n2	—
C	—	n9

B needs n2. A will change n2 first.

START A AND C TOGETHER



Keep the original order for conflicting calls. Independent calls can compute together.

If earlier work changes the query, inspect again.

The node set found before execution is valid only while the query's inputs stay unchanged.

INSPECTION FINDS NO MATCHES



Result: []

AN EARLIER ABILITY ADDS AN EDGE



Edge



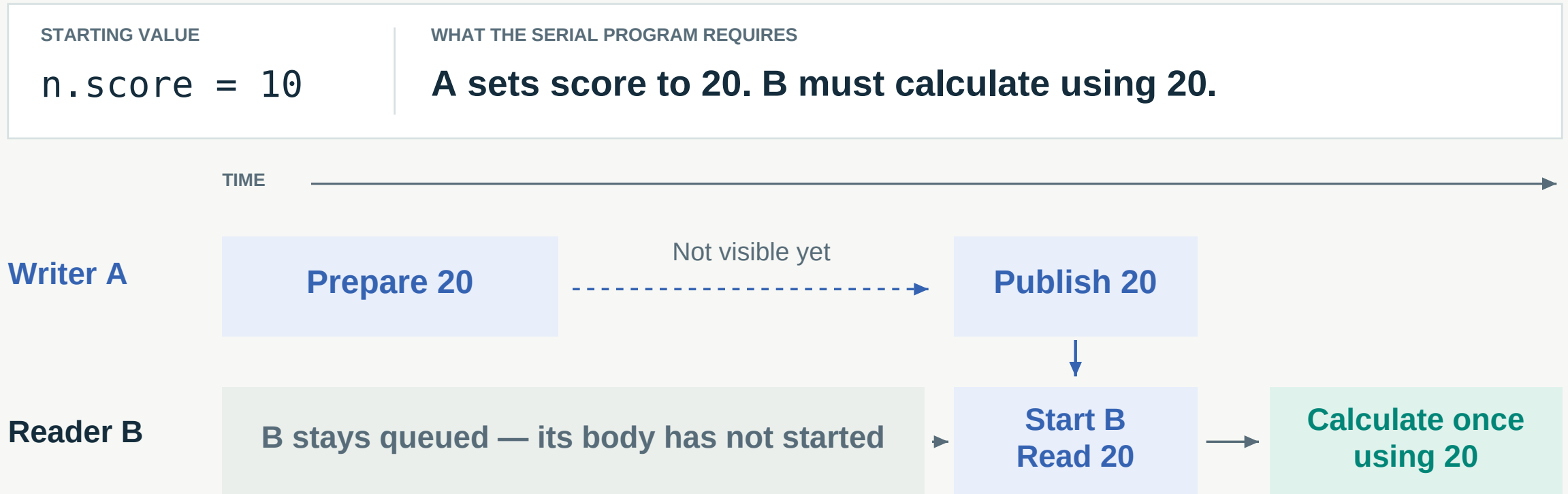
Result: [n4]

Before B starts: query again and update its dependencies to include n4.

Track changes to the searched edges and filter inputs, even when the result is empty.

B waits before starting, then calculates once.

Before either body starts, inspection finds: A writes n; B reads n.



No calculation with 10. No discarded first attempt.

Refresh inspection if its inputs changed. Keep final validation for unproven accesses.

The hypothesis is a measurable trade-off.

Inspection is useful when avoiding wasted computation saves more time than inspection costs.

Potential benefit

Fewer failed speculative attempts
Workers start independent calls first

Potential cost

Extra lookups and re-inspection
Conservative sets may delay safe work

CONTROLLED COMPARISON

Serial

Current speculation

Eager memory tracking

Inspect then schedule

Vary: shared nodes, query size, computation cost, and graph changes.

Measure: total time, discarded computation, inspection cost, and unnecessary waits.

Start with one query and one falsifiable claim.

Can finding dependencies before launch reduce total execution time?

01

Inspect simple queries

Known starting node.
One-hop graph queries.
Summarize reads and writes.

02

Schedule before launch

Hold back conflicting calls.
Keep ordered effects.
Retain speculative validation.

03

Measure the full cost

Check identical behavior.
Include inspection overhead.
Find when it pays off.

The contribution to establish

**Use graph syntax to generate dependency inspection,
then schedule ability calls before their bodies run.**

Inspection before execution has clear precedents.

The candidate contribution is automatic extraction from Jac, with its ordered walker behavior.

Runtime preprocessing

Saltz, Berryman & Wu · 1991

Compiler-generated runtime preprocessing can help expose parallel work.

Galois

Kulkarni et al. · PLDI 2007

Abstractions and runtime conflict detection expose parallelism in irregular programs.

Legion

Bauer et al. · SC 2012

Logical regions and access privileges provide dependence information for task scheduling.

Inspect first, execute second. Study what Jac can extract automatically, and at what cost.

The current prototype is the experimental baseline.

Existing native walker speculation provides an execution substrate and a serial reference.

Implemented

Opt-in speculative entry abilities
Buffered effects and ordered commit
Serial fallback for unsupported effects

Proposed

Pre-execution query inspection
and dependency-aware scheduling

Baseline measurement: 2 warm-ups, 7 timed runs; native AOT + RC; 16 logical CPUs.
Pre-execution inspection remains unevaluated. Source: PR #9215 and bundled benchmark JSON.

EXISTING CPU MICROBENCHMARK		
Mode	Median	Speedup
Serial / off	424 ms	1.00×
2 threads	341 ms	1.24×
4 threads	223 ms	1.90×
16 independent tasks; identical output; 0 fallbacks.		